Compiler Design

GATE CSE 2027 · complete formula, rule, method & trap sheet

75 reference boxes · **9** chapters ·

Compiler Phases & Lexical Analysis · Top-Down Parsing (LL) · Bottom-Up Parsing (LR family) · Syntax-Directed Translation · Runtime Environments · Intermediate Code Generation · Code Optimisation & Code Generation · Problem → Standard Method Playbook · Last-minute one-liners

1 Compiler Phases & Lexical Analysis

The phases of a compiler — Core

NOTES

A compiler translates a source program into target code in a sequence of phases. The first three phases **analyse** the source (the **front end**); the rest **synthesise** the target program (the **back end**). The **symbol table** and **error handler** are used by every phase.

FLOW

Source code → Lexical analysis → Syntax analysis → Semantic analysis → Intermediate code gen → Code optimisation → Code generation → Target code

REFERENCE TABLE

Phase	Input → Output	Typical errors caught
Lexical analysis	characters → **tokens**	illegal character, unterminated string/comment
Syntax analysis	tokens → **parse tree**	missing `;` or `)`, malformed statement
Semantic analysis	parse tree → annotated tree	type mismatch, undeclared variable, wrong argument count
Intermediate code gen	annotated tree → three-address code	—
Code optimisation	IR → better IR	—
Code generation	IR → machine / assembly code	—

GATE FAVOURITE

GATE angle

"Which phase detects this error?" is a favourite. Remember: **lexer** = bad characters/tokens, **parser** = bad grammar structure, **semantic analyser** = bad meaning (types, declarations). Also: a **pass** is one traversal over the whole program and may bundle several phases; a **phase** is a logical step.

Tokens, patterns, lexemes — Core

REFERENCE TABLE

Term	Meaning	Example
Lexeme	actual character sequence in the source	`count`, `3.14`, `>=`
Token	(token-name, attribute) pair given to the parser	(id, ptr to symbol table), (num, 3.14), (relop, GE)
Pattern	rule describing the lexemes of a token, usually a regular expression	`letter (letter|digit)*` for identifiers

NOTES

Token patterns are **regular languages**, so a lexical analyser is built as a finite automaton: regular expressions → NFA → DFA → minimised DFA. Tools like **Lex/Flex** automate this.

Two disambiguation rules

RULES & FACTS

1. **Longest match (maximal munch):** the lexer always consumes the longest prefix that matches some pattern.
2. **Priority:** if two patterns match the same longest lexeme (e.g. `if` as keyword vs identifier), the pattern listed **first** wins — so keywords are listed before the identifier rule.

WORKED EXAMPLE

Worked numerical — counting tokens

Q: How many tokens are in `if (a >= b) x = a-1;` ?
A: Split by maximal munch:

```
if  (  a  >=  b  )  x  =  a  -  1  ;
 1  2  3  4   5  6  7  8  9  10 11 12   → 12 tokens
```

Note `>=` is **one** token (longest match), not `>` and `=`.

WORKED EXAMPLE

Worked numerical — maximal munch trap

Q: Tokenise `a+++b` in C.
A: `a`, then the longest match from `+++b` is `++`, then `+`, then `b`:

```
a   ++   +   b   → 4 tokens   (parsed as (a++) + b)
```

A string literal such as `"i=%d"` is always a **single** token, however many characters it holds.

Input buffering & the symbol table — High-yield

NOTES

Reading character by character is slow, so lexers use a **two-buffer scheme** with a **sentinel** (eof) at the end of each half: one comparison per character normally, and an extra check only when the sentinel is hit. The lexer also enters each new identifier into the **symbol table**; the token's attribute is a pointer to that entry. Whitespace and comments are stripped here and never reach the parser.

COMMON TRAP

What the lexer cannot do

It sees one token at a time with no notion of nesting, so it cannot check balanced brackets, a missing semicolon, or types. That is why syntax needs a grammar (context-free), not just regular expressions.

2 Top-Down Parsing (LL)

Preparing a grammar for top-down parsing — Core

KEY POINT

Remove left recursion (a top-down parser would loop forever)

```
A → Aα | β
⇓
A  → β A'
A' → α A' | ε
```

KEY POINT

Left-factor (so one lookahead can choose)

```
A → αβ1 | αβ2
⇓
A  → α A'
A' → β1 | β2
```

FIRST and FOLLOW Core

RULES & FACTS

- **FIRST(X)**: set of terminals that can begin a string derived from X (include ε if X ⇒* ε).
- **FOLLOW(A)**: set of terminals that can appear immediately to the right of A in some sentential form. FOLLOW(start) always contains `$`. ε is never in FOLLOW.

NOTES

FOLLOW rules for each production `B → α A β`: add FIRST(β) \{ε} to FOLLOW(A); if β is empty or β ⇒* ε, add FOLLOW(B) to FOLLOW(A).

WORKED EXAMPLE

Worked numerical — FIRST, FOLLOW and the LL(1) table

Q: Grammar (the left-recursion-free expression grammar):

```
E  → T E'
E' → + T E' | ε
T  → F T'
T' → * F T' | ε
F  → ( E ) | id
```

A:

```
FIRST(E) = FIRST(T) = FIRST(F) = { (, id }
FIRST(E') = { +, ε }        FIRST(T') = { *, ε }

FOLLOW(E)  = { $, ) }
FOLLOW(E') = FOLLOW(E)                = { $, ) }
FOLLOW(T)  = FIRST(E')\ε ∪ FOLLOW(E') = { +, $, ) }     (E' is nullable)
FOLLOW(T') = FOLLOW(T)                = { +, $, ) }
FOLLOW(F)  = FIRST(T')\ε ∪ FOLLOW(T)  = { *, +, $, ) }
```

Place `A → α` in cell M[A, a] for every a ∈ FIRST(α); if α ⇒* ε also for every a ∈ FOLLOW(A):

	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

No cell has two entries → the grammar is **LL(1)**.

The LL(1) condition Core

NOTES

A grammar is LL(1) iff for every pair of productions `A → α | β`:

RULES & FACTS

1. FIRST(α) ∩ FIRST(β) = ∅.
2. At most one of α, β can derive ε.
3. If β ⇒* ε, then FIRST(α) ∩ FOLLOW(A) = ∅.

NOTES

Equivalently: **every cell of the parsing table has at most one entry**.

COMMON TRAP

Must-know facts

• Left-recursive, ambiguous, or non-left-factored grammars are **never** LL(1).
• Every LL(1) grammar is unambiguous and is also LR(1) — but not the other way round.
• LL(1) parsers need no backtracking; they use an explicit **stack** (start with `$ S`) and the table.

WORKED EXAMPLE

Worked numerical — a grammar that is NOT LL(1)

Q: The dangling-else grammar `S → i E t S S' | a, S' → e S | ε, E → b`. Is it LL(1)?
A: FOLLOW(S') = FOLLOW(S) = { e, $ } (because S → iEtS**S'** and S appears before S' giving e). The production S' → ε goes under FOLLOW(S') which contains `e`, but S' → eS also sits in M[S', e]. Two entries in one cell → **not LL(1)**.

Parse trace (table-driven) Core

NOTES

Input `id + id $` with the table above (stack top on the left):

FORMULA / WORKING

```
Stack        Input      Action
E $          id+id $    E → T E'
T E' $       id+id $    T → F T'
F T' E' $    id+id $    F → id
id T' E' $   id+id $    match id
T' E' $      +id $      T' → ε
E' $         +id $      E' → + T E'
+ T E' $     +id $      match +
T E' $       id $       T → F T'
F T' E' $    id $       F → id
id T' E' $   id $       match id
T' E' $      $          T' → ε
E' $         $          E' → ε
$            $          ACCEPT
```

3 Bottom-Up Parsing (LR family)

Shift-reduce parsing Core

NOTES

A stack holds grammar symbols; the parser repeatedly does one of four things: **shift** (push next input token), **reduce** (replace a **handle** on top of the stack by the LHS of its production), **accept**, or **error**.

RULES & FACTS

- **Handle**: the substring that matches the RHS of a production and whose reduction is one step of the reverse rightmost derivation.
- **Viable prefix**: a prefix of a right-sentential form that can appear on the stack (never extends past the handle).
- **Conflicts**: **shift-reduce** (both legal) or **reduce-reduce** (two reductions legal). Ambiguous grammars always produce conflicts.

WORKED EXAMPLE

Worked numerical — finding the handle

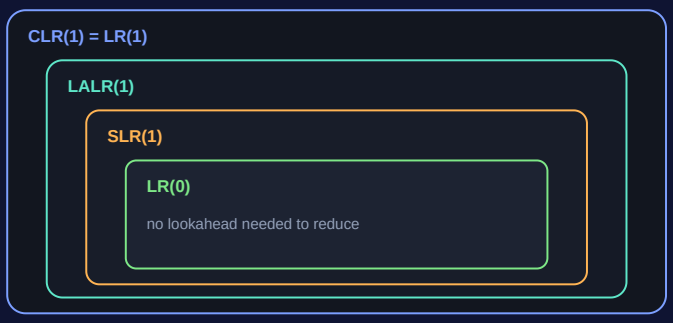
Q: Grammar `E → E+T | T, T → T*F | F, F → id`. Find the handle of the right-sentential form `E + T * id`.
A: Reverse the rightmost derivation: the rightmost symbol `id` was produced by `F → id`, so reduce it first.

```
E + T * id   ⇐ (reduce F→id)  E + T * F
E + T * F    ⇐ (reduce T→T*F) E + T
E + T        ⇐ (reduce E→E+T) E
```

Handle of `E + T * id` = **id** (reducing by F → id).

Power of the LR parsers Core

DIAGRAM



REFERENCE TABLE

Parser	Items	Reduce A→α when lookahead ∈	No. of states
LR(0)	LR(0) items	every symbol (no lookahead)	n
SLR(1)	LR(0) items	FOLLOW(A)	n (same as LR(0))
LALR(1)	LR(1) items, states with same core merged	merged lookahead set	n (same as LR(0))
CLR(1)	LR(1) items	exact lookahead set of the item	≥ n (largest)

GATE FAVOURITE

GATE angle

- LR(0), SLR(1) and LALR(1) tables have the **same number of states**; CLR(1) has the same or more.
- Merging LR(1) states into LALR can create **reduce-reduce** conflicts, but never a new **shift-reduce** conflict.
- No ambiguous grammar is LR(k) for any k. LL(1) ⊂ LR(1), but an LL(1) grammar need not be SLR(1).
- YACC/Bison generate **LALR(1)** parsers; they resolve S/R by shifting and R/R by the earlier production.

Building the LR(0) automaton Core

NOTES

An **item** is a production with a dot marking how much has been seen (`A → α·β`). **Closure** adds `B → ·γ` for every nonterminal B right after a dot; **goto(I, X)** moves the dot over X. Always **augment** the grammar with `S' → S` first.

WORKED EXAMPLE

Worked numerical — canonical LR(0) states

Q: Build the LR(0) automaton for `(1) S → AA, (2) A → aA, (3) A → b`.
A:

```
I0: S'→·S   S→·AA   A→·aA   A→·b
I1 = goto(I0,S): S'→S·                          (accept on $)
I2 = goto(I0,A): S→A·A   A→·aA   A→·b
I3 = goto(I0,a): A→a·A   A→·aA   A→·b
I4 = goto(I0,b): A→b·
I5 = goto(I2,A): S→AA·
I6 = goto(I3,A): A→aA·
goto(I2,a)=I3, goto(I2,b)=I4, goto(I3,a)=I3, goto(I3,b)=I4
```

7 states (I0–I6). The same grammar has **10 states** as a CLR(1) automaton, because states I3, I4, I6 each split into two copies with different lookaheads ({a,b} vs {$}).

NOTES

SLR(1) table — FOLLOW(S)={$}, FOLLOW(A)={a,b,$}:

REFERENCE TABLE

State	ACTION			GOTO	
	a	b	$	S	A
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

NOTES

Trace on `abb$` (stack holds alternating symbol/state):

FORMULA / WORKING

```
Stack          Input   Action
0              abb$    shift 3
0 a 3          bb$     shift 4
0 a 3 b 4      b$      reduce A→b   (pop 2, goto(3,A)=6)
0 a 3 A 6      b$      reduce A→aA  (pop 4, goto(0,A)=2)
0 A 2          b$      shift 4
0 A 2 b 4      $       reduce A→b   (pop 2, goto(2,A)=5)
0 A 2 A 5      $       reduce S→AA  (pop 4, goto(0,S)=1)
0 S 1          $       ACCEPT
```

NOTES

A reduction by `A → α` pops **2·|α|** stack entries (symbol + state each), or just |α| states if only states are stored.

When SLR(1) is not enough Core

NOTE

Classic example

Grammar: `S → L = R | R, L → *R | id, R → L`

```
FOLLOW(S) = {$}      FOLLOW(L) = {=, $}      FOLLOW(R) = {=, $}

State after seeing L at the start:   S → L· = R      R → L·
 • shift on '='                      (from S → L·=R)
 • reduce R→L if '=' ∈ FOLLOW(R)  ✔  → shift-reduce conflict on '='
```

Not SLR(1) — FOLLOW is too coarse. But the true lookahead for `R → L·` in this state is only `$`, so the grammar is **LALR(1) and CLR(1)** (CLR: 14 states, LALR: 10).

COMMON TRAP

Another gotcha

`S → AaAb | BbBa, A → ε, B → ε` is **LL(1)** but **not SLR(1)**: after the empty prefix, both `A→ε` and `B→ε` reduce on {a,b} (FOLLOW sets), a reduce-reduce conflict. With exact LR(1) lookaheads (A on `a`, B on `b`) it is conflict-free.

Operator-precedence parsing (quick note) Extension

NOTES

A simpler bottom-up method for **operator grammars** (no ε-productions, no two adjacent nonterminals on any RHS). It uses precedence relations `⋖ ≐ ⋗` between terminals only, which makes it fast but less general than LR.

4 Syntax-Directed Translation

SDD vs SDT Core

RULES & FACTS

- **Syntax-Directed Definition (SDD)**: a CFG with semantic rules attached to productions — it says *what* to compute, not the order.
- **Syntax-Directed Translation scheme (SDT)**: a CFG with **program fragments (actions) embedded** in the RHS — the position of an action fixes *when* it runs.

Synthesized vs inherited attributes Core

KEY POINT

Synthesized

Value of a node is computed from its **children** (and the node's own attributes). Flows **bottom-up**.

```
E → E1 + T   E.val = E1.val + T.val
```

KEY POINT

Inherited

Value of a node comes from its **parent and/or siblings**. Flows **top-down / sideways**.

```
D → T L   L.type = T.type
```

REFERENCE TABLE

Class	Rule	Evaluation
S-attributed	only synthesized attributes	bottom-up; fits LR parsing (evaluate at reduce time)
L-attributed	inherited attribute of a RHS symbol may depend only on the parent's inherited attributes and symbols **to its left** (never to the right, never its own synthesized attributes)	single left-to-right depth-first traversal; fits LL parsing

GATE FAVOURITE

GATE angle

Every **S-attributed** SDD is also **L-attributed** (it has no inherited attributes at all), but not vice versa. The **dependency graph** among attributes must be **acyclic** for any valid evaluation order to exist.

WORKED EXAMPLE

Worked numerical — evaluating a synthesized attribute

Q: SDD: `E → E1 + T {E.val = E1.val + T.val} | T {E.val = T.val}`, `T → T1 * F {T.val = T1.val × F.val} | F`, `F → digit {F.val = digit.lexval}`. Compute E.val for `3 + 4 * 5`.

A: The grammar gives `*` higher precedence, so the tree is E → E + T with the right T → T * F:

```
F.val = 3   → T.val = 3   → E1.val = 3
F.val = 4, F.val = 5 → T.val = 4 × 5 = 20
E.val = E1.val + T.val = 3 + 20 = 23
```

SDT for translation — infix to postfix Core

NOTES

Place the `print` action at the **end** of the RHS (a **postfix SDT**); it then runs exactly when the production is reduced, so it works with bottom-up parsers:

FORMULA / WORKING

```
E → E + T   { print('+') }
E → T
T → T * F   { print('*') }
T → F
F → num     { print(num.val) }
```

WORKED EXAMPLE

Worked numerical

Q: What is printed for input `2 + 3 * 4 + 5`?

A: Parse as `(2 + (3*4)) + 5`. Actions fire in reduction order:

```
2   3   4   *   +   5   +      → output:  2 3 4 * + 5 +
```

COMMON TRAP

Rule of thumb

Actions at the **end** of every production behave like a **post-order** walk of the parse tree. An action placed in the middle of a RHS runs after the symbols to its left have been parsed — in an LR parser this is handled by introducing an ε-**marker** nonterminal.

Inherited attribute example — declarations Core

FORMULA / WORKING

```
D → T L          { L.type = T.type }
T → int          { T.type = integer }
L → L1 , id      { L1.type = L.type;  addType(id.entry,
L.type) }
L → id           { addType(id.entry, L.type) }
```

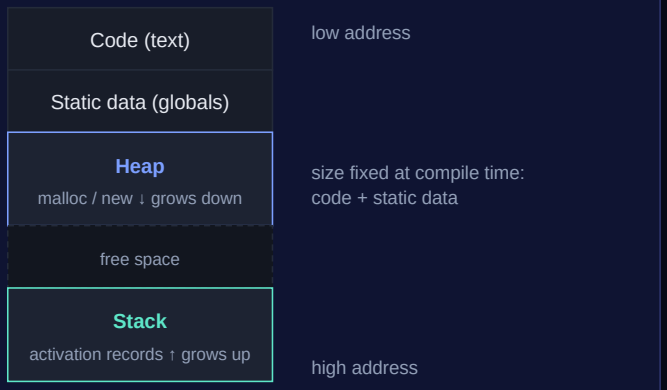
NOTES

For `int a, b, c` : T.type = integer is passed **down** into L, then copied left-to-right along the list, so every identifier is entered in the symbol table with type integer. L.type is an **inherited** attribute — this SDD is L-attributed but **not** S-attributed.

5 Runtime Environments

Storage organisation Core

DIAGRAM



NOTES

Code and static data have sizes known at compile time. The **stack** (procedure calls) and **heap** (dynamic allocation) grow toward each other into the free space. The textbook figure usually draws heap growing up and stack growing down; what matters is that they grow **towards each other**.

Activation record (stack frame) Core

NOTES

Each procedure call pushes one **activation record**; it is popped on return. Typical fields, top to bottom:

REFERENCE TABLE

Field	Purpose
Actual parameters	arguments passed by the caller
Returned value	space for the function's result
Control link (dynamic link)	points to the **caller's** activation record
Access link (static link)	points to the activation record of the **lexically enclosing** procedure — used to reach non-local variables
Saved machine status	return address, saved registers
Local data & temporaries	local variables and compiler temporaries

NOTES

Because every call gets its own record, **recursion** works: each invocation has independent locals. Stack depth equals the number of live (not yet returned) calls.

WORKED EXAMPLE

Worked numerical — access links

Q: In a language with nested procedures, a procedure at nesting depth 4 refers to a variable declared in an enclosing procedure at depth 2. How many access links must be followed?

A: Follow `4 − 2 = 2` access links (each hop climbs one lexical level).

Static vs dynamic scoping Core

RULES & FACTS

- **Static (lexical) scoping**: a non-local name is resolved by the **textual nesting** of the source. Used by C, Java, Python, Pascal.
- **Dynamic scoping**: a non-local name is resolved by the **chain of callers** at run time (follow control links).

WORKED EXAMPLE

Worked numerical — what is printed?

```
int x = 10;
void f()  { print(x); }
void g()  { int x = 20; f(); }
main()    { g(); }
```

Static scoping: f's `x` is the global (f is written outside g) → prints **10**.
Dynamic scoping: f was called by g, so the nearest `x` in the call chain is g's → prints **20**.

Parameter passing Core

REFERENCE TABLE

Mechanism	What is passed	Effect
Call by value	copy of the value	callee cannot change caller's variable
Call by reference	address of the actual	formal is an alias; changes visible to caller immediately
Copy-restore (value-result)	copy in, then copy back on return	changes visible only at return
Call by name	the **expression** itself, re-evaluated at every use	like textual substitution (macro)

WORKED EXAMPLE

Worked numerical — four mechanisms, one program

```
int i = 1;
void P(int x, int y) { x = x + 1;  y = y * 2; }
main: P(i, i);   print(i);
```

```
Value          : x, y are copies; i stays
→ 1
Reference      : x, y both alias i: i=1+1=2, then i=2*2=4
→ 4
Copy-restore   : x=1,y=1 → x=2, y=2; restore x then y: i=2,
i=2  → 2
Name           : x and y both mean i, same as reference
→ 4
```

Copy-restore depends on the **order** of copying back (assumed left to right here).

COMMON TRAP

Reference vs name

They differ when the actual is an expression whose variables change inside the callee — e.g. `A[i]` . Reference fixes the address **at call time**; name re-evaluates `A[i]` with the **current** i at every use (see solved problem 6).

6 Intermediate Code Generation

Three-address code (TAC) Core

NOTES

Each instruction has **at most one operator on the right-hand side** and at most three addresses: `x = y op z` . Complex expressions are broken up using compiler-generated **temporaries**.

WORKED EXAMPLE

Worked numerical — expression to TAC

Q: Write TAC for `a = b + c * d - e` .

A:

```
t1 = c * d
t2 = b + t1
a  = t2 - e          → 3 instructions, 2 temporaries (t1,
t2)
```

Representations

REFERENCE TABLE

Form	Fields	Notes
Quadruples	(op, arg1, arg2, result)	result is an explicit name → easy to move instructions when optimising
Triples	(op, arg1, arg2)	no result field; a result is referred to by the **position** of its triple → saves space but reordering is hard
Indirect triples	triples + a separate list of pointers	reorder by changing the pointer list, not the triples

WORKED EXAMPLE

Worked numerical — the three forms

Q: Represent `a = b * -c + b * -c`.

A:

```
Quadruples
   op      arg1  arg2  result
0  uminus  c           t1
1  *       b     t1    t2
2  uminus  c           t3
3  *       b     t3    t4
4  +       t2    t4    t5
5  =       t5          a          → 6 quadruples

Triples (positions are the names)
0  uminus  c
1  *       b     (0)
2  uminus  c
3  *       b     (2)
4  +       (1)   (3)
5  =       a     (4)
```

Quadruples need 4 fields × 6 rows, triples 3 fields × 6 rows. A DAG would share `b * -c` and need only 4 operations.

Array address calculation Core

NOTES

For an array with element width *w* and base address *base* (indices from 0):

KEY POINT

1-D

```
addr(A[i]) = base + i·w
```

KEY POINT

2-D, row-major, A[R][C]

```
addr(A[i][j]) = base + (i·C + j)·w
```

KEY POINT

2-D, column-major

```
addr(A[i][j]) = base + (j·R + i)·w
```

KEY POINT

Lower bounds l1, l2 (1-based etc.)

```
row-major: base + ((i−l1)·C + (j−l2))·w
```

WORKED EXAMPLE

Worked numerical

Q: `int A[10][20]`, base = 1000, w = 4. Find the address of `A[3][5]` if (a) 0-indexed row-major, (b) 0-indexed column-major, (c) 1-indexed row-major.

A:

```
(a) 1000 + (3·20 + 5)·4 = 1000 + 260 = 1260
(b) 1000 + (5·10 + 3)·4 = 1000 + 212 = 1212
(c) 1000 + ((3−1)·20 + (5−1))·4 = 1000 + 176 = 1176
```

Basic blocks and the control-flow graph Core

NOTES

A **basic block** is a maximal straight-line sequence with a single entry (the first instruction) and a single exit (the last). Find the blocks with the **leader** rules:

RULES & FACTS

1. The first instruction of the program is a leader.
2. Any instruction that is the **target** of a jump is a leader.
3. Any instruction **immediately after** a (conditional or unconditional) jump is a leader.

NOTES

Each block runs from a leader up to (not including) the next leader. The **flow graph** has an edge B→C if C can follow B: C's leader is the jump target of B's last instruction, or C directly follows B and B does not end in an unconditional jump.

WORKED EXAMPLE

Worked numerical — leaders, blocks, edges

```
1: i = 0
2: s = 0
3: if i >= n goto 9
4: t = i % 2
5: if t == 0 goto 7
6: s = s + i
7: i = i + 1
8: goto 3
9: return s
```

Targets: 9, 7, 3. After jumps: 4 (after 3), 6 (after 5), 9 (after 8). Leaders = {1, 3, 4, 6, 7, 9}.

```
B1 = {1,2}   B2 = {3}   B3 = {4,5}   B4 = {6}   B5 = {7,8}
B6 = {9}
Edges: B1→B2, B2→B3, B2→B6, B3→B4, B3→B5, B4→B5, B5→B2
```

6 basic blocks, 7 edges. The loop is B2 → B3 → (B4) → B5 → B2.

Backpatching Core

NOTES

In one pass, jump targets of **forward** jumps (boolean expressions, `if`, `while`) are not yet known. The compiler emits the jump with an empty target, records it in a **list** (`truelist` / `falselist`), and later `backpatch(list, label)` fills in the address once it is known.

7 Code Optimisation & Code Generation

Machine-independent optimisations Core

REFERENCE TABLE

Technique	Before	After
Constant folding	`x = 3 * 4`	`x = 12`
Constant propagation	`x = 5; y = x + 1`	`y = 6`
Common subexpression elimination	`a = b+c; d = b+c`	`a = b+c; d = a`
Copy propagation	`x = y; z = x + 1`	`z = y + 1`
Dead-code elimination	assignment whose value is never used	removed
Strength reduction	`x * 2`, `x * 8`	`x << 1`, `x << 3`
Loop-invariant code motion	`while(..){ t = a*b; ...}`	`t = a*b; while(..){...}`
Induction-variable elimination	`j = 4*i` recomputed every iteration	`j = j + 4`
Loop unrolling / jamming	many small iterations	fewer jumps, less overhead

NOTES

Peephole optimisation slides a small window (a few instructions) over the **target** code, replacing patterns with cheaper ones: redundant load/store removal, unreachable-code removal, algebraic simplification (`x+0` , `x*1`), flow-of-control tweaks (jump to a jump).

DAG of a basic block Core

NOTES

A DAG for a basic block has one node per distinct value; identical computations share a node, which exposes **common subexpressions** and **dead code** automatically.

WORKED EXAMPLE

Worked numerical — DAG construction

Q: Build the DAG for `a = b + c; b = a − d; c = b + c; d = a − d` and count its nodes.

A:

```
Leaves (initial values): b0, c0, d0                    → 3 nodes
a = b0 + c0              → node N1 (+)  labelled a
b = a − d0 = N1 − d0     → node N2 (−)  labelled b
c = b + c0 = N2 + c0     → node N3 (+)  labelled c   (b changed, so NOT the same as N1)
d = a − d0 = N1 − d0     → same as N2 → label d is added to N2, no new node

Total nodes = 3 leaves + 3 internal = 6
Optimised code:  a = b + c;  d = a − d;  c = d + c   (b = d)
→ 3 operations, not 4
```

Data-flow analysis Core

NOTES

For each basic block B, compute `IN[B]` and `OUT[B]` by iterating the equations until nothing changes (a **fixed point**). Three classic problems:

REFERENCE TABLE

Problem	Direction	Meet	Equations
Reaching definitions	forward	∪ (may)	OUT[B] = gen[B] ∪ (IN[B] − kill[B]); IN[B] = ∪ OUT[pred]
Live variables	backward	∪ (may)	IN[B] = use[B] ∪ (OUT[B] − def[B]); OUT[B] = ∪ IN[succ]
Available expressions	forward	∩ (must)	OUT[B] = e_gen[B] ∪ (IN[B] − e_kill[B]); IN[B] = ∩ OUT[pred]

COMMON TRAP

Memory hook

May analyses (reaching defs, liveness) use **union** and start with ∅. **Must** analyses (available expressions) use **intersection** and start all non-entry blocks with the universal set. Reaching defs & available exprs go **forward**; liveness goes **backward**.

WORKED EXAMPLE

Worked numerical — reaching definitions

```
B1:  d1: a = 1       d2: b = 2
B2:  d3: a = a + b          (B2 loops back to itself, then falls to B3)
B3:  d4: c = a + b
Edges: B1→B2, B2→B2, B2→B3
```

gen: B1={d1,d2}, B2={d3}, B3={d4}. kill: B1={d3}, B2={d1}, B3=∅ (d1 and d3 both define a).

```
OUT[B1] = {d1,d2}
Iteration:  IN[B2] = OUT[B1] ∪ OUT[B2] = {d1,d2} ∪ OUT[B2]
            OUT[B2] = {d3} ∪ (IN[B2] − {d1})
  pass 1:   IN[B2] = {d1,d2}            OUT[B2] = {d3, d2}
  pass 2:   IN[B2] = {d1,d2,d3}         OUT[B2] = {d3, d2}  (stable)
IN[B3]  = OUT[B2] = {d2, d3}            OUT[B3] = {d2, d3, d4}
```

So at B3, the definition **d1 does not reach** (killed by d3); only d3's value of `a` reaches. That justifies using d3 for constant/copy propagation decisions.

WORKED EXAMPLE

Worked numerical — live variables

```
B1: a = ..; b = ..      →  B2 and B3
B2: c = a + b           →  B4
B3: c = a * b           →  B4
B4: print(c)
```

```
B4: use={c}  def=∅          IN[B4]={c}   OUT[B4]=∅
B2: use={a,b} def={c}       OUT[B2]=IN[B4]={c}        IN[B2]={a,b}
B3: use={a,b} def={c}       OUT[B3]={c}               IN[B3]={a,b}
B1: use=∅   def={a,b}       OUT[B1]=IN[B2]∪IN[B3]={a,b}
IN[B1]=∅
```

IN[B1] = ∅ means no variable is used before being defined — a good sign. A variable assigned but not in OUT is **dead** and its assignment can be eliminated.

Code generation & register allocation Extension

RULES & FACTS

- Code generation chooses instructions, **allocates registers**, and orders the code. Variables in registers are much faster than memory.
- **Graph colouring**: one node per variable (live range); an edge between two variables that are **live at the same time**; with k registers, colour the graph with k colours. If it cannot be k-coloured, some variable is **spilled** to memory.
- Minimum registers needed (no spills) = **chromatic number** of the interference graph ≥ maximum number of simultaneously live variables.

WORKED EXAMPLE

Worked numerical — registers needed

```
1: a = 1
2: b = 2
3: c = a + b
4: d = a * c
5: e = b + d
6: print(e)
```

```
Live after each instruction:  1:{a}  2:{a,b}  3:{a,b,c}  4:
{b,d}  5:{e}  6:ø
Interference edges: a–b, a–c, b–c, b–d
```

a, b, c form a triangle, so at least 3 colours are needed, and 3 suffice (d can reuse a's or c's register). **Minimum registers = 3**. Note that `d` and `a` do not interfere because `a` 's last use is in instruction 4 itself, the one that defines `d` .

8 Problem → Standard Method Playbook

1. Type checking & semantic analysis High-yield

METHOD

Recognition: the question asks whether an expression, assignment, function call, array access, or operator application is type-correct.
Method: (1) annotate each identifier with its declared type, (2) propagate expression types bottom-up, (3) apply operator compatibility and implicit-conversion rules, (4) check assignment/return/argument compatibility, (5) distinguish compile-time semantic errors from runtime errors.
Trap: syntax can be valid while semantics are invalid; parsing alone does not establish type correctness.

WORKED EXAMPLE

Worked problem — type checking

Q: Suppose `f` has type `int (float, int)` , `x` is `int` , and `y` is `float` . Is `f(x, y)` type-correct if implicit widening `int→float` is allowed but `float→int` is not?

METHOD NOTES

Solution: The first argument `x:int` widens to `float` . The second argument `y:float` cannot narrow to the required `int` . Therefore the call is **not type-correct**.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- FIRST(α) is the set of terminals that can begin strings derived from α; include ε when α can derive ε.
- Dom(entry)={entry}; for other nodes iterate Dom(n)={n}∪⋂Dom(pred) to a fixed point.

LAST-MINUTE · MUST REMEMBER

- Lexical analysis converts characters into tokens; lexical errors are detected before parsing.
- Longest-match (maximal munch) is the standard lexer rule; keyword recognition may be handled after identifier matching.
- An LL(1) grammar must avoid FIRST/FIRST conflicts and FIRST/FOLLOW conflicts for ε-productions.
- LR parsing is bottom-up and constructs a rightmost derivation in reverse.
- Shift-reduce conflicts and reduce-reduce conflicts are the two classic LR parsing conflicts.
- Syntax-directed definitions attach semantic rules/actions to grammar productions.
- Three-address code typically represents expressions using simple instructions with at most three addresses.
- Common optimisations include constant folding, common-subexpression elimination, dead-code elimination, and strength reduction.
- **Type checking is semantic analysis:** parse trees can be syntactically valid but type-invalid.
- A back edge u→v exists when v dominates u; the natural-loop header is v.
- Reference counting cannot reclaim cycles; tracing collectors use reachability from roots.
- Assembler makes object code; linker resolves cross-object symbols; loader places the executable in memory.
- Panic-mode recovery discards input to a synchronizing token; phrase-level recovery performs a local correction.

LAST-MINUTE · COMMON TRAPS

- SLR, canonical LR(1), and LALR differ mainly in the information used to resolve reductions and the number of states.
- Optimisation must preserve program semantics; a locally faster transformation is not valid if observable behaviour changes.
- Unreachable memory is a leak; a dangling pointer points to storage whose lifetime has ended.

LAST-MINUTE · GATE FAVOURITES

- Know activation records, static/dynamic links, parameter passing, and storage allocation for runtime-environment questions.